hdoc day 46 by trishil chauhan

C Programming | Bubble Sort and First/Last Occurrences

Objective: Read a variable-length integer array, sort it in ascending order using **Bubble_sort()**, and use **occurrence()** to find the first and last positions of a user-entered target. Arrays are passed using pointers.

Complete C Program

```
#include <stdio.h>

void Bubble_sort(int *nums, int n) {
    for (int i = 0; i < n - 1; i++) {
        int swapped = 0;
        for (int j = 0; j < n - i - 1; j++) {
            if (*(nums + j) > *(nums + j + 1)) {
                int temp = *(nums + j);
                *(nums + j) = *(nums + j + 1);
                *(nums + j + 1) = temp;
                swapped = 1;
            }
        }
        if (!swapped) break;
    }
}

void occurrence(const int *nums, int n) {
    int target, first = -1, last = -1;
    printf("Enter target integer: ");
    if (scanf("%d", &target) != 1) {
        printf("Invalid target.\n");
        return;
    }
    for (int i = 0; i < n; i++) {
        if (*(nums + i) == target) {
            if (first == -1) first = i;
            last = i;
        }
    }
    if (first == -1) {
        printf("First occurrence (value, index): -1, -1\n");
        printf("Last occurrence  (value, index): -1, -1\n");
    } else {
        printf("First occurrence (value, index): %d, %d\n", *(nums + first), first);
        printf("Last occurrence  (value, index): %d, %d\n", *(nums + last), last);
    }
}

int main(void) {
    int n;
    printf("Enter number of elements: ");
    if (scanf("%d", &n) != 1 || n <= 0 || n > 10000) {
        printf("Invalid array size (1 to 10000 required).\n");
        return 1;
    }
    int nums[n];
    printf("Enter %d integers: ", n);
    for (int i = 0; i < n; i++) {
        if (scanf("%d", nums + i) != 1) {
            printf("Invalid integer input.\n");
            return 1;
        }
```

```
    }
    Bubble_sort(nums, n);
    printf("Sorted array: ");
    for (int i = 0; i < n; i++) printf("%d ", *(nums + i));
    printf("\n");
    occurrence(nums, n);
    return 0;
}
```

Sample Output - Target Present

```
Enter number of elements: 7
Enter 7 integers: 5 2 8 2 1 5 2
Sorted array: 1 2 2 2 5 5 8
Enter target integer: 2
First occurrence (value, index): 2, 1
Last occurrence  (value, index): 2, 3
```

Sample Output - Target Absent

```
Enter number of elements: 4
Enter 4 integers: 4 1 7 3
Sorted array: 1 3 4 7
Enter target integer: 9
First occurrence (value, index): -1, -1
Last occurrence  (value, index): -1, -1
```

Explanation

Bubble_sort() compares adjacent elements and swaps them when they are out of ascending order. occurrence() scans the sorted array: it records the first matching index once and updates the last matching index for every match. Indices are zero-based. When no match exists, both value/index pairs are -1, -1.